
12.5 Selection

As important as it is, sorting is not the only interesting problem dealing with a total order relation on a set of elements. There are a number of applications in which we are interested in identifying a single element in terms of its rank relative to the sorted order of the entire set. Examples include identifying the minimum and maximum elements, but we may also be interested in, say, identifying the *median* element, that is, the element such that half of the other elements are smaller and the remaining half are larger. In general, queries that ask for an element with a given rank are called *order statistics*.

Defining the Selection Problem

In this section, we discuss the general order-statistic problem of selecting the k^{th} smallest element from an unsorted collection of n comparable elements. This is known as the *selection* problem. Of course, we can solve this problem by sorting the collection and then indexing into the sorted sequence at index $k - 1$. Using the best comparison-based sorting algorithms, this approach would take $O(n \log n)$ time, which is obviously an overkill for the cases where $k = 1$ or $k = n$ (or even $k = 2$, $k = 3$, $k = n - 1$, or $k = n - 5$), because we can easily solve the selection problem for these values of k in $O(n)$ time. Thus, a natural question to ask is whether we can achieve an $O(n)$ running time for all values of k (including the interesting case of finding the median, where $k = \lfloor n/2 \rfloor$).

12.5.1 Prune-and-Search

We can indeed solve the selection problem in $O(n)$ time for any value of k . Moreover, the technique we use to achieve this result involves an interesting algorithmic design pattern. This design pattern is known as *prune-and-search* or *decrease-and-conquer*. In applying this design pattern, we solve a given problem that is defined on a collection of n objects by pruning away a fraction of the n objects and recursively solving the smaller problem. When we have finally reduced the problem to one defined on a constant-sized collection of objects, we then solve the problem using some brute-force method. Returning back from all the recursive calls completes the construction. In some cases, we can avoid using recursion, in which case we simply iterate the prune-and-search reduction step until we can apply a brute-force method and stop. Incidentally, the binary search method described in Section 5.1.3 is an example of the prune-and-search design pattern.

12.5.2 Randomized Quick-Select

In applying the prune-and-search pattern to finding the k^{th} smallest element in an unordered sequence of n elements, we describe a simple and practical algorithm, known as **randomized quick-select**. This algorithm runs in $O(n)$ *expected* time, taken over all possible random choices made by the algorithm; this expectation does not depend whatsoever on any randomness assumptions about the input distribution. We note though that randomized quick-select runs in $O(n^2)$ time in the *worst case*, the justification of which is left as an exercise (R-12.25). We also provide an exercise (C-12.56) for modifying randomized quick-select to define a **deterministic** selection algorithm that runs in $O(n)$ *worst-case* time. The existence of this deterministic algorithm is mostly of theoretical interest, however, since the constant factor hidden by the big-Oh notation is relatively large in that case.

Suppose we are given an unsorted sequence S of n comparable elements together with an integer $k \in [1, n]$. At a high level, the quick-select algorithm for finding the k^{th} smallest element in S is similar to the randomized quick-sort algorithm described in Section 12.2.1. We pick a “pivot” element from S at random and use this to subdivide S into three subsequences L , E , and G , storing the elements of S less than, equal to, and greater than the pivot, respectively. In the prune step, we determine which of these subsets contains the desired element, based on the value of k and the sizes of those subsets. We then recur on the appropriate subset, noting that the desired element’s rank in the subset may differ from its rank in the full set. Pseudocode for randomized quick-select is shown in Code Fragment 12.8.

Algorithm quickSelect(S, k):

Input: Sequence S of n comparable elements, and an integer $k \in [1, n]$

Output: The k^{th} smallest element of S

if $n == 1$ **then**

return the (first) element of S .

pick a random (pivot) element x of S and divide S into three sequences:

- L , storing the elements in S less than x
- E , storing the elements in S equal to x
- G , storing the elements in S greater than x

if $k \leq |L|$ **then**

return quickSelect(L, k)

else if $k \leq |L| + |E|$ **then**

return x

{each element in E is equal to x }

else

return quickSelect($G, k - |L| - |E|$)

{note the new selection parameter}

Code Fragment 12.8: Randomized quick-select algorithm.

12.5.3 Analyzing Randomized Quick-Select

Showing that randomized quick-select runs in $O(n)$ time requires a simple probabilistic argument. The argument is based on the **linearity of expectation**, which states that if X and Y are random variables and c is a number, then

$$E(X + Y) = E(X) + E(Y) \quad \text{and} \quad E(cX) = cE(X),$$

where we use $E(\mathcal{Z})$ to denote the expected value of the expression $\mathcal{Z}$.

Let $t(n)$ be the running time of randomized quick-select on a sequence of size n . Since this algorithm depends on random events, its running time, $t(n)$, is a random variable. We want to bound $E(t(n))$, the expected value of $t(n)$. Say that a recursive call of our algorithm is “good” if it partitions S so that the size of each of L and G is at most $3n/4$. Clearly, a recursive call is good with probability at least $1/2$. Let $g(n)$ denote the number of consecutive recursive calls we make, including the present one, before we get a good one. Then we can characterize $t(n)$ using the following **recurrence equation**:

$$t(n) \leq bn \cdot g(n) + t(3n/4),$$

where $b \geq 1$ is a constant. Applying the linearity of expectation for $n > 1$, we get

$$E(t(n)) \leq E(bn \cdot g(n) + t(3n/4)) = bn \cdot E(g(n)) + E(t(3n/4)).$$

Since a recursive call is good with probability at least $1/2$, and whether a recursive call is good or not is independent of its parent call being good, the expected value of $g(n)$ is at most the expected number of times we must flip a fair coin before it comes up “heads.” That is, $E(g(n)) \leq 2$. Thus, if we let $T(n)$ be shorthand for $E(t(n))$, then we can write the case for $n > 1$ as

$$T(n) \leq T(3n/4) + 2bn.$$

To convert this relation into a closed form, let us iteratively apply this inequality assuming n is large. So, for example, after two applications,

$$T(n) \leq T((3/4)^2 n) + 2b(3/4)n + 2bn.$$

At this point, we should see that the general case is

$$T(n) \leq 2bn \cdot \sum_{i=0}^{\lceil \log_{4/3} n \rceil} (3/4)^i.$$

In other words, the expected running time is at most $2bn$ times a geometric sum whose base is a positive number less than 1. Thus, by Proposition 4.5, $T(n)$ is $O(n)$.

Proposition 12.7: *The expected running time of randomized quick-select on a sequence S of size n is $O(n)$, assuming two elements of S can be compared in $O(1)$ time.*